```
    Note.new("G"),
    Note.new("Bb"),
    Note.new("Db"),
    Note.new("E")
  ]
)

File.open("posterity", "w+") do |f|
  Marshal.dump(c, f)
end
```

Finally, our friends read it in and are transported by our creation's beauty:

```
chord = Marshal.load(File.open("posterity"))
chord.play               # => "G-Bb-Db-E"
```

Custom Serialization Strategy

Not all objects can be dumped: bindings, procedure objects, instances of class IO, and singleton objects cannot be saved outside the running Ruby environment (a TypeError will be raised if you try). Even if your object doesn't contain one of these problematic objects, you may want to take control of object serialization yourself.

Marshal provides the hooks you need. In the objects that require custom serialization, simply implement two instance methods: one called marshal_dump, which writes the object out to a string, and one called marshal_load, which reads a string that you had previously created and uses it to initialize a newly allocated object.

The instance method marshal_dump should return an object representing the state to be dumped. When the object is subsequently reconstituted using load, the method marshal_load will be called with this object and will use it to set the state of its receiver—it will be run in the context of an allocated but not initialized object of the class being loaded.

For instance, here is a sample class that defines its own serialization. For whatever reasons, Special doesn't want to save one of its internal data members, @volatile. The author has decided to serialize the two other instance variables in an array:

ospace/marshal_load.rb

```
class Special
  def initialize(valuable, volatile, precious)
    @valuable = valuable
    @volatile = volatile
    @precious = precious
  end

  def marshal_dump
    [@valuable, @precious]
  end

  def marshal_load(variables)
    @valuable = variables[0]
    @precious = variables[1]
    @volatile = "unknown"
  end

  def to_s
    "#{@valuable} #{@volatile} #{@precious}"
```